API JIRA for Tickets | API JIRA per Ticket

Project Overview / Panoramica del Progetto

English

API JIRA for Tickets is a Windows desktop application built in Rust that monitors JIRA issues in real-time. It periodically fetches open tickets from a Jira REST API, compares them with previously stored states, detects changes (new tickets, status changes, squad reassignments), and sends notifications via Telegram when modifications are detected. This tool is designed for teams that need automated alerts about ticket lifecycle events without manually checking JIRA.

The application runs as a background service (every 60 seconds), maintains audit logs, and provides localized user feedback in Italian and English.

Italiano

API JIRA per Ticket è un'applicazione desktop Windows sviluppata in Rust che monitora i ticket JIRA in tempo reale. Recupera periodicamente i ticket aperti da un'API REST di Jira, li confronta con stati precedentemente memorizzati, rileva cambiamenti (nuovi ticket, cambi di stato, cambio di squadra) e invia notifiche tramite Telegram quando vengono rilevate modifiche. Questo strumento è progettato per team che hanno bisogno di avvisi automatizzati sugli eventi del ciclo di vita dei ticket senza controllare manualmente JIRA.

L'applicazione viene eseguita come servizio in background (ogni 60 secondi), mantiene log di audit e fornisce feedback agli utenti localizzati in italiano e inglese.

Tech Stack / Tecnologie Utilizzate

Component	Technology	Version	Purpose
Language	Rust	2024 Edition	Core runtime and async processing
Web Framework	Axum	0.8.4	HTTP server infrastructure
Async Runtime	Tokio	1.46.1	Async task orchestration and scheduling
HTTP Client	Reqwest	0.12.22	JIRA and Telegram API communication
Serialization	Serde + Serde JSON	1.0+	Data model serialization/deserialization
UI Dialogs	RFD (rfd-crate)	0.15.3	Native file/message dialogs
System Tray	Tray-Icon	0.21.1	Windows system tray integration
Localization	sys-locale	0.3.2	System locale detection (IT/EN)
Logging	log + simple-logging	0.4.28, 2.0.2	Application logging to file
Windows Resources	winres	0.1.12	Windows resource compilation (.ico)

Architecture & Folder Structure / Architettura e Struttura Cartelle

Folder Tree Overview

```
src/
├── main.rs                          # Application entry point
├── main_single.rs                    # Alternative single-run entry point
├── command/
│   ├── mod.rs
│   └── get_issues_api.rs           # Core business logic: API parsing & ticket
comparison
├── controller/
│   ├── mod.rs
│   ├── get_issues.rs               # JIRA API integration & HTTP requests
│   └── send_message_to_bot.rs      # Telegram notification dispatcher
├── models/
│   ├── mod.rs
│   ├── user_data/
│   │   ├── mod.rs
│   │   └── user.rs                 # User credential management
│   └── all_open_issues/
│       ├── mod.rs
│       ├── issues_response.rs      # JIRA API response DTOs
│       └── coverted_issues_response.rs  # Custom issue representation
├── constants/
│   ├── mod.rs
│   ├── common_costants.rs          # Localization strings & common values
│   └── request_constants.rs        # API endpoints, headers, JQL queries
└── utils.rs                         # UI helper functions (dialogs)
```

Architectural Layers

Layer	Modules	Responsibility
Entry Point	`main.rs`	Initialize logger, spawn monitoring loop (60s intervals)
Controller Layer	`controller/`	Handle external API calls (JIRA, Telegram)
Command/Orchestration	`command/get_issues_api.rs`	Parse API responses, compare states, detect changes
Data Models	`models/`	Define request/response DTOs and custom data structures
Configuration	`constants/`	Store API endpoints, JQL queries, localized strings
Utilities	`utils.rs`	Display UI notifications and error dialogs

Key Design Decisions

1. **Modular Structure**: Separation of concerns between API controllers, business logic, and data models enables maintenance and testing.
2. **Async Architecture**: Tokio-based async/await pattern allows efficient polling without blocking the UI thread.
3. **Localization First**: `MsgKey` enum + language modules enable dual-language support at the constant level.
4. **File-based State**: `issues_oldest.json` serves as the comparison baseline—persisted locally to detect deltas across runs.
5. **Error Resilience**: Retry logic with exponential backoff for Telegram notifications (max 3 retries).

Core Logic Flow / Flusso della Logica Principale

Data Flow Diagram

```
┌──────────────────────────────────────────────────────────────┐
│ main.rs: Loop (every 60 seconds)                             │
└──────────────────────────────────────────────────────────────┘
                    │
                    │
                    ▼
┌──────────────────────────────────────────────────────────────┐
│ controller/get_issues.rs: api_get_issues()                   │
│ • Load user credentials from user_data.json                  │
│ • Call JIRA REST API (POST with Basic Auth)                  │
│ • Return raw JSON response                                    │
└──────────────────────────────────────────────────────────────┘
                    │
                    │
                    ▼
┌──────────────────────────────────────────────────────────────┐
│ command/get_issues_api.rs: use_api_get_issues()              │
│ • serialize_api_response() → Parse JSON into JiraSearchResponse │
│ • iterate_on_response() → Extract key, status, squad, timestamp │
│ • convert_my_response_to_json_string() → Serialize to JSON   │
│ • compare_glbam_tickets() → Compare NEW vs OLD states        │
└──────────────────────────────────────────────────────────────┘
                    │
                    │
      ╔═══════════╩═══════════╗
      ╔═══════════╗           ╔═══════════╗
      │           │           │           │
      ▼           ▼           ▼           ▼

┌──────────────────┐    ┌──────────────────────────┐
│ No Changes       │    │ Changes Detected         │
│ Update JSON file │    │ • Build change list      │
│ Continue loop    │    │ • Log changes            │
└──────────────────┘    │ • Show UI notification   │
                        │ • Send Telegram message  │
                        │ • Update JSON file       │
                        └──────────────────────────┘
                                  │
                                  ▼
```

```
                    ┌──────────────────────────────┐
                    │ controller/send_message_to_ │
                    │ bot.rs: send_message_to_    │
                    │ bot_func()                  │
                    │ • POST to Telegram API      │
                    │ • Retry 3x with backoff     │
                    │ • Log success/failure       │
                    └──────────────────────────────┘
```

Change Detection Logic

The comparison flow detects three types of changes per ticket:

1. **New Tickets**: Ticket exists in new response but not in `issues_oldest.json`
2. **Status Changes**: `glbam_status` differs between current and previous state
3. **Squad Changes**: `glbam_squad` differs between current and previous state
4. **Timestamp Updates**: `updated` field has changed

Each change is logged, displayed in a UI dialog, and formatted into a Telegram message.

Development Guidelines / Linee Guida di Sviluppo

Code Style & Conventions

Aspect	Convention	Example
Naming	Snake_case for functions/variables, CamelCase for types	`serialize_api_response()`, `struct ConvertedIssues`
Async	Use `pub async fn` for I/O operations; spawn tasks with `tokio::spawn()`	See `controller/` layer
Error Handling	Return `Result<T, E>`; use `.unwrap_or_else()` with localized panic messages	`use_api_get_issues()` returns `Result<(), String>`
Logging	Use `log::info!()`, `log::error!()` for audit trails	All decisions logged to `app_issues.log`
Comments	Use emojis + clear descriptions for function documentation	📦 Purpose, 📝 🎯 Description, 📥 Parameters, 📤 Returns
Constants	Define in `constants/` modules; use `MsgKey` enum for UI strings	`request_constants.rs`, `common_costants.rs`
DTOs	Use Serde `#[derive(Serialize, Deserialize)]` for API models	`JiraSearchResponse`, `User`

Adding New Features

When adding a new JIRA field extraction:

1. Add the field to the corresponding custom field struct in `models/all_open_issues/issues_response.rs`
2. Add an extractor function in `command/get_issues_api.rs` (following the `get_status()` / `get_squad()` pattern)
3. Update the `ConvertedIssues` struct in `coverted_issues_response.rs`
4. Add comparison logic in `compare_glbam_tickets()` if change detection is needed
5. Add localized message keys in `common_costants.rs` for notifications

When adding localized strings:

1. Add new key to `MsgKey` enum in `common_costants.rs`
2. Define translations in `pub mod it { }` and `pub mod en { }` blocks
3. Add pattern match in `localized()` function for both languages

Error Handling Strategy

- **API Errors**: Log and display UI dialog; for Telegram, retry with backoff
- **File I/O Errors**: Panic with localized message (fail-fast for critical I/O)
- **Deserialization Errors**: Log full error context; panic to prevent processing inconsistent data
- **Missing Custom Fields**: Return default values (`NO_SQUAD`, `NO_STATUS`) instead of panicking

Testing & Validation

- **Unit Testing**: Not currently implemented; consider adding for comparison and parsing logic
- **Manual Testing**: Run `cargo build --release` and test against live JIRA/Telegram APIs
- **Load Testing**: Verify behavior with large ticket counts (current max: 500 from JQL query)

Setup & Commands / Setup e Comandi

Prerequisites / Prerequisiti

- **Rust**: Install via rustup (2024 edition or later)
- **Cargo**: Included with Rust installation
- **Windows OS**: Application is Windows-specific (uses `windows_subsystem = "windows"`)
- **JIRA Account**: Valid credentials with read access to GLBAM project
- **Telegram Bot**: Active bot token and target chat ID

Installation & Build

```
# Clone the repository
git clone <repository-url>
cd api_jira_for_tickets

# Install dependencies (automatic via Cargo)
cargo build --release

# Output binary location
./target/release/api_jira_for_tickets.exe
```

First-Run Configuration

On first run, the application creates:

- **Directory**: `ApiJiraForTickets/` (in the current working directory)
- **File**: `ApiJiraForTickets/user_data.json` (template with placeholders)

Example `user_data.json`:

```
{
  "user": "username_CHANGE_ME",
  "password": "INSERT_YOUR_PASSWORD_HERE"
}
```

Steps to configure:

1. Update `user` with your JIRA username
2. Update `password` with your JIRA API token or password
3. Update JIRA endpoint in `constants/request_constants.rs` if needed
4. Update Telegram bot credentials in `constants/request_constants.rs`:
    - `BOT_ID`: Your Telegram bot token
    - `CHAT_ID_VALUE`: Target chat ID for notifications

Running the Application

```
# Run in debug mode (with console output)
cargo run

# Run release binary directly
./target/release/api_jira_for_tickets.exe

# View logs
cat app_issues.log            # Unix/WSL
type app_issues.log           # Windows CMD
```

Key Files Generated at Runtime

File	Purpose
`app_issues.log`	Application audit log (info and error messages)
`issues_oldest.json`	Snapshot of latest JIRA tickets (used for delta comparison)
`ApiJiraForTickets/user_data.json`	User credentials (JIRA auth)

Building for Distribution

```
# Create optimized release build
cargo build --release

# The executable includes Windows icon from build.rs
# Icon source: src/assets/computer2.ico
```

Development Commands

```
# Format code
cargo fmt

# Check for issues
cargo clippy

# Run in debug mode with logging
RUST_LOG=debug cargo run

# Clean build artifacts
cargo clean
```

Additional Notes / Note Aggiuntive

Security Considerations

- **Credentials**: User data is stored in plaintext in `user_data.json`. Consider implementing encryption or using system credential manager.
- **API Tokens**: Telegram bot token and chat ID should ideally be loaded from environment variables or secure vaults.
- **JIRA Custom Fields**: The hardcoded custom field IDs (`customfield_10001`, `customfield_11708`) are specific to the GLBAM Jira instance and must be updated when migrating to different JIRA installations.

Performance Notes

- **Polling Interval**: 60 seconds is hardcoded in `main.rs`. Adjust based on API rate limits and notification frequency requirements.
- **Max Results**: JQL query fetches max 500 tickets. For larger backlogs, implement pagination or filtering.
- **Memory**: Application maintains in-memory HashMap for comparison; suitable for typical ticket counts.

Future Enhancement Opportunities

1. Implement SQLite database instead of JSON file for persistent state
2. Add configuration UI for credentials and poll intervals
3. Support additional notification channels (Slack, email, webhooks)
4. Implement custom user filtering (project, assignee, priority)
5. Add unit tests for deserialization and comparison logic
6. Migrate to config files (TOML/YAML) for external configuration

---

Last Updated: 2026 | Revision: 1.0